Сессионное задание.

Выполнил: Капров Александр Николаевич М26-545.

Вопрос №1. Опишите трехсторонний протокол распределения ключей между парами участников с использованием третьей стороны.

Проблема: как двум участникам при симметричном шифровании безопасно передать друг другу общий секретный ключ, чтобы его никто не перехватил?

Решение: использование Центра распределения ключей (KDC).

Роли участников:

1. KDC (Центр распределения ключей) – это доверенная третья сторона (то есть выделенный сервер аутентификации), которая хранит секретные ключи всех пользователей и умеет создать временные ключи для защиты их общения.
2. Участник №1 – инициатор процесса обмена, который обладает секретным ключом. Этот ключ знает только участник №1 и KDC.
3. Участник №2 – получатель данных, который обладает секретным ключом. Этот ключ знает только участник №2 и KDC.

Основные этапы:

1. Участник №1 генерирует случайное проверочное число и отправляет в KDC запрос.
2. KDC генерирует общий сеансовый ключ для участников и отправляет Участнику №1 ответ, который состоит из двух зашифрованных частей. Первую часть KDC шифрует личным ключом Участника №1 (внутрь кладут сеансовый ключ и проверочное число). Вторую часть шифрует личным ключом Участника №2. (внутрь кладут тот же сеансовый ключ и имя Участника №1).
3. Участник №1 расшифровывает первую часть своим личным ключом, сверяет проверочное число и забирает сеансовый ключ. Вторую зашифрованную часть, он прочитать не может, поэтому отправляет ее Участнику №2.
4. Участник №2 расшифровывает вторую часть своим личным ключом, достает оттуда тот же сеансовый ключ и видит имя Участника №1.

Итог: теперь у Участника №1 и Участника №2 единый сеансовый ключ для защиты своих данных. Злоумышленник не сможет перехватить этот ключ, так как он передавался только в зашифрованном виде на личных ключах участников.

Вопрос №2. Приведите формальные модели шифров.

В криптографии любой шифр описывается как модель, которая состоит из 5 компонентов.

$$\Sigma = (X, Y, K, E, D)$$

- X – это исходный текст (сообщения), который мы хотим зашифровать.
- Y – это зашифрованный текст (криптограмма), который получается на выходе после шифрования.
- K – это секретные ключи. Их назначение – сделать шифрование уникальным для каждого пользователя.
- E – это алгоритмы, которые берут текст, смешивают его с конкретным ключом и выдают на выходе зашифрованный текст.
- D – это алгоритмы, которые выполняют обратную операцию, то есть берут зашифрованный текст, применяют нужный ключ и возвращают исходное сообщение в прежний вид.

Главное условие работы модели: если мы зашифровали исходный текст с помощью алгоритма зашифрования на конкретном ключе, то при расшифровании соответствующим алгоритмом на этом же ключе мы обязаны получить обратно точно такой же исходный текст без единого искажения.

Вопрос №3. Приведите принципы построения поточных систем шифрования.

– Принцип детерминированности: алгоритм, который создает гамму (то есть случайные символы), должен работать по четким формулам, это нужно для того, чтобы отправитель и получатель, используя одинаковый секретный ключ, получили одинаковую гамму. Если бы алгоритм выдавал каждый раз случайные числа, то получатель бы просто не смог расшифровать текст, потому что гамма не совпала бы с гаммой отправителя.

– Принцип псевдослучайности: гамма должна состоять из случайных чисел, в ней должно быть равное количество нулей и единиц, а комбинации не должны повторяться, это нужно для того, чтобы злоумышленник не мог найти закономерности.

– Принцип смешивания: наложение гаммы на исходный текст должно происходить через логическое сложение по модулю 2 (XOR).

– Принцип независимости: знаки гаммы должны создаваться отдельно и никак не зависеть от шифруемого текста.

Вопрос №4. Приведите пример режима шифрования, в котором биективность всех шифрующих отображений базового режима не является необходимой для однозначного расшифрования криптограмм. Ответ обоснуйте.

Пример: режим счетчика симметричных блочных шифров.

В этом режиме при расшифровании не нужно запускать алгоритм в обратную сторону. Потому, что в режиме счетчика базовый алгоритм шифра, например AES вообще не используется для зашифрования исходного текста напрямую. Вместо этого он генерирует случайные числа, то есть гаммы. И поскольку, как при зашифровании, так и при расшифровании алгоритм использует одну и ту же функцию зашифрования для создания этой гаммы, нам без разницы умеет ли алгоритм работать в обратную сторону. Расшифрование происходит не за счет обратимости шифра, а благодаря битовой операции XOR.

Механизм расшифрования:

1. Получатель берет число счетчика и прогоняет его через алгоритм зашифрования.
2. Алгоритм выдает гаммы, какой был у отправителя.
3. Получатель накладывает эту гамму через операцию XOR на зашифрованный текст и возвращает сообщение в прежний вид.

Вопрос №5. Сколько 64-битовых блоков открытого текста искажается при расшифровании криптограммы симметричного блочного шифра, если в шифрованном тексте исказился один бит и используется режим гаммирования с обратной связью по шифртексту, в котором блоки открытого и шифрованного текста имеют длину 8 бит? Ответ обоснуйте.

Логика работы режима CFB-8: в этом режиме текст обрабатывается кусочками по 8 бит. Программа хранит в памяти историю из последних 64 бит зашифрованного текста. Алгоритм шифрует эту историю, берет первые 8 бит результата и прогоняет через XOR с открытым текстом. Получившийся кусочек записывается в историю и заменяет старые данные, то есть здесь прошлый зашифрованный текст напрямую влияет на то какая гамма создастся на следующем шаге.

Как распространяется ошибка на блоки открытого текста: если при передаче в зашифрованном тексте испортился один бит. В 8-битном кусочке, где исказился бит, при расшифровании тоже испортится один бит. Потом этот кусочек испорченный записывается в историю памяти. Пока он находится в истории, алгоритм шифрования будет выдавать

неправильную гамму. За один шаг в историю заходит по 8 бит, то есть ошибка будет портить гамму на протяжении 64/8 = 8 шагов. На 9-м шаге происходит самовосстановление, то есть новые данные вытеснят испорченный кусок из истории и дальше текст будет нормально расшифровываться.

Расчеты: из-за одного испорченного бита у нас пострадало 1 + 8 = 9 блоков по 8 бит, то есть это 9 * 8 = 72 бита испорченных данных, блоки имеют размер 64 бита, то есть наш 72 бита испорченных данных это один целый блок 64 бит плюс еще 8 бит, которые залезут на соседний блок. Если блок поврежден хотя бы немного, то он считается уничтоженным, получается из-за одной ошибки будут полностью испорчены и уничтожены 2 блока открытого текста.

Вопрос №6. Однонаправленная хэш-функция принимает значения из множества двоичных n-мерных векторов. При каком n случайная пара сообщений составляет коллизию с вероятностью, не большей 10-9? Обязательно должен быть приведен ход решения.

Если длина хэша равна n бит, то всего можно получить 2^n вариантов разных хэш-кодов. Когда мы берем одну случайную пару сообщений, шанс того, что их хэши совпадут равен 1/2^n. Записываем неравенство, по условию этот шанс совпадения должен быть не больше, чем 10^-9. Неравенство такое: 1/2^n <= 10^-9. Переворачиваем дроби 2^n >= 10^9. Потом чтобы вытащить n из степени используем логарифм по основанию 2, то есть n >= log2(10^9), выносим девятку из степени, получается n >= 9 * log2(10), переводим логарифм в число получается n >= 9 * 3.3219 и умножаем n >= 29.897. Округляем получается n = 30.